DO NOW — RECALL

Quicksort: tracing and implementation

Name: _____ Class: _____ Date: _____

1. After partitioning [8,1,7,3,6] around pivot 6, where is the pivot?

2. Why does recursive quicksort need a base case?

3. What is the worst-case time complexity of merge sort?

4. If the pivot finishes at index 3, give left and right recursive bounds in an array 0..6.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Index 2 in [1,3,6,8,7].

2. Model answer

To stop on empty or single-element partitions.

3. Model answer

O(n log n).

4. Model answer

Left 0..2, right 4..6; do not include pivot index 3 again.

Vocabulary — know these terms

quicksort — Recursive sort using partitioning around a pivot.

base case — Condition ending recursion.

partition index — Final index occupied by the pivot.

worst case — Input arrangement causing the most expensive run.

recursion stack — Storage used for nested calls.

NEW LEARNING

Read, annotate and explain

Stop small subarrays

When low>=high there are zero or one elements to sort: return immediately.

Partition then divide

Partition places the pivot at its final index p. Do not recursively include index p.

Sort two sides independently

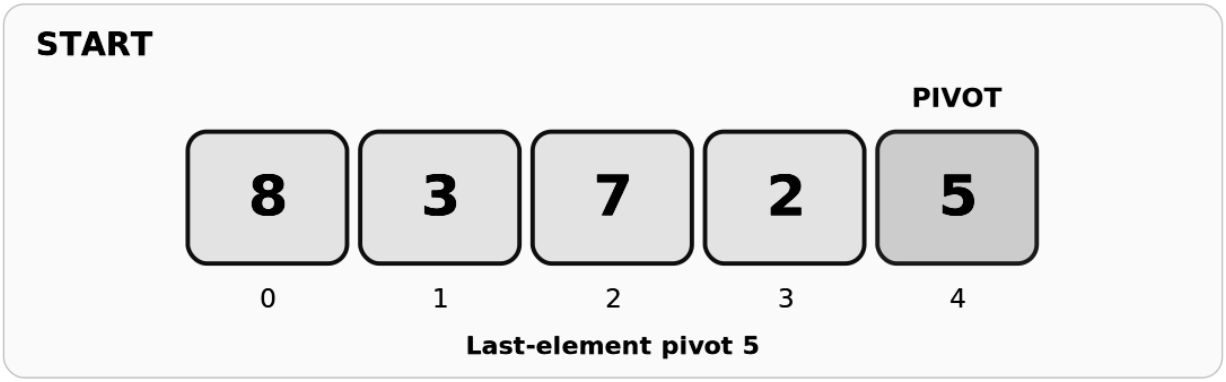
Call QuickSort on low..p−1 and p+1..high. The pivot itself stays fixed.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

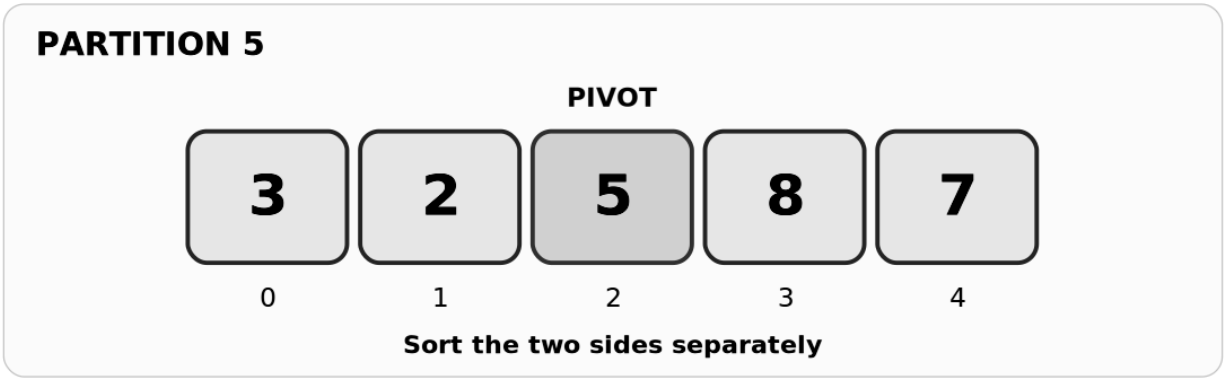
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



START: 8, 3, 7, 2, 5. Last-element pivot 5

New Learning

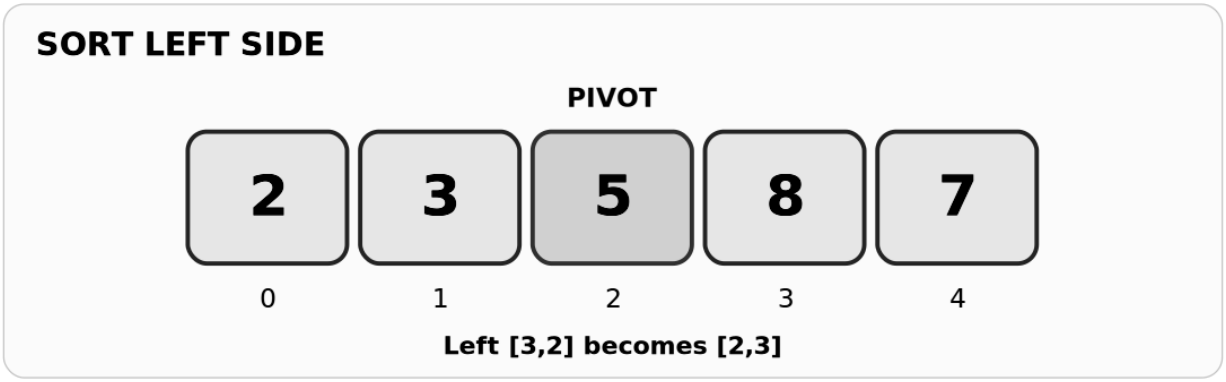


PARTITION 5: 3, 2, 5, 8, 7. Sort the two sides separately

New Learning · original lesson diagrams

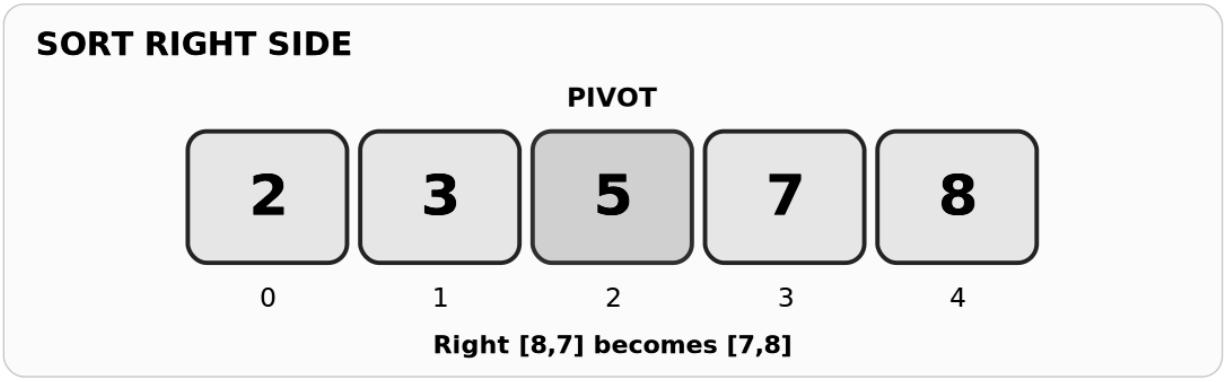
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



SORT LEFT SIDE: 2, 3, 5, 8, 7. Left [3,2] becomes [2,3]

New Learning



SORT RIGHT SIDE: 2, 3, 5, 7, 8. Right [8,7] becomes [7,8]

I DO — WORKED EXAMPLE

Follow the worked method

Trace recursive quicksort on [5,1,7,3] (last pivot).

Predict the next state

Partition around 3, moving 1 to the left and placing pivot 3 at index 1.

Reveal and explain

Left segment [1] is already sorted; sort only the right segment [7,5].

Reveal and explain

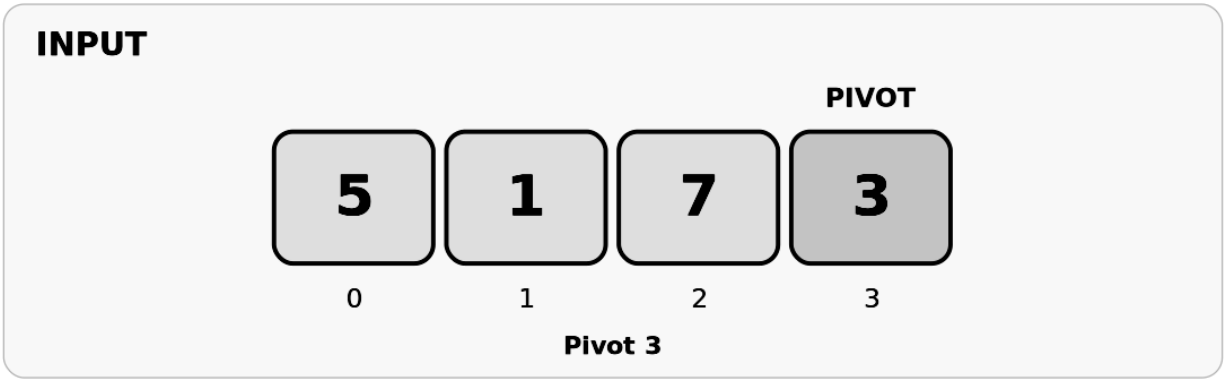
The right recursion produces [5,7]; final array [1,3,5,7].

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace recursive quicksort on [5,1,7,3] (last pivot).

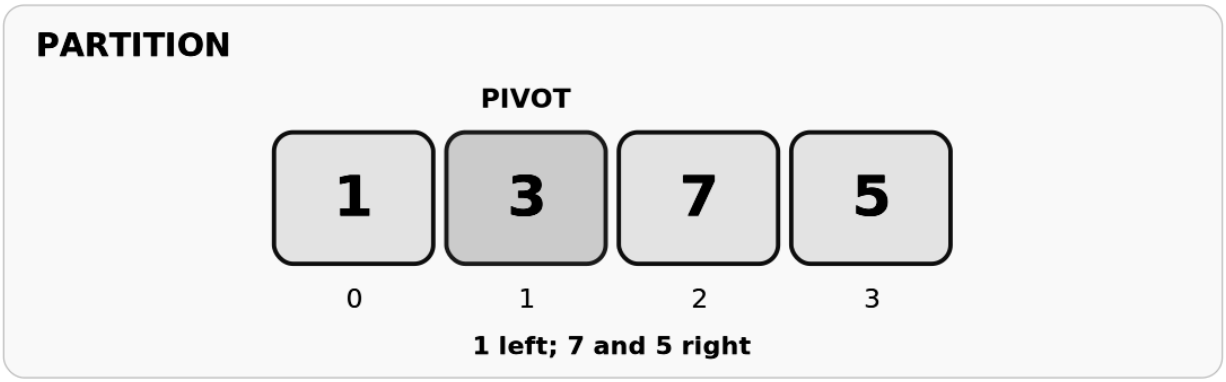
Partition around 3, moving 1 to the left and placing pivot 3 at index 1.



INPUT: 5, 1, 7, 3. Pivot 3

Trace recursive quicksort on [5,1,7,3] (last pivot).

Partition around 3, moving 1 to the left and placing pivot 3 at index 1.



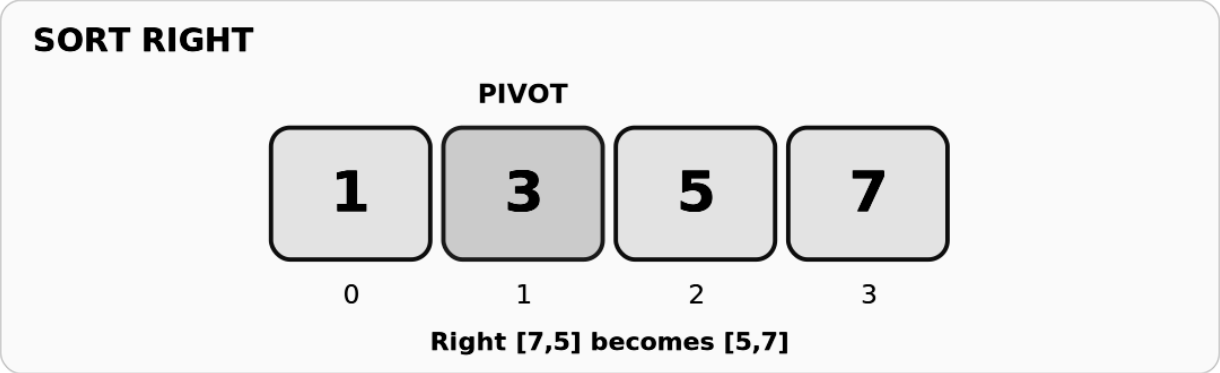
PARTITION: 1, 3, 7, 5. 1 left; 7 and 5 right

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace recursive quicksort on [5,1,7,3] (last pivot).

Partition around 3, moving 1 to the left and placing pivot 3 at index 1.



SORT RIGHT: 1, 3, 5, 7. Right [7,5] becomes [5,7]

WE DO — GUIDED PRACTICE

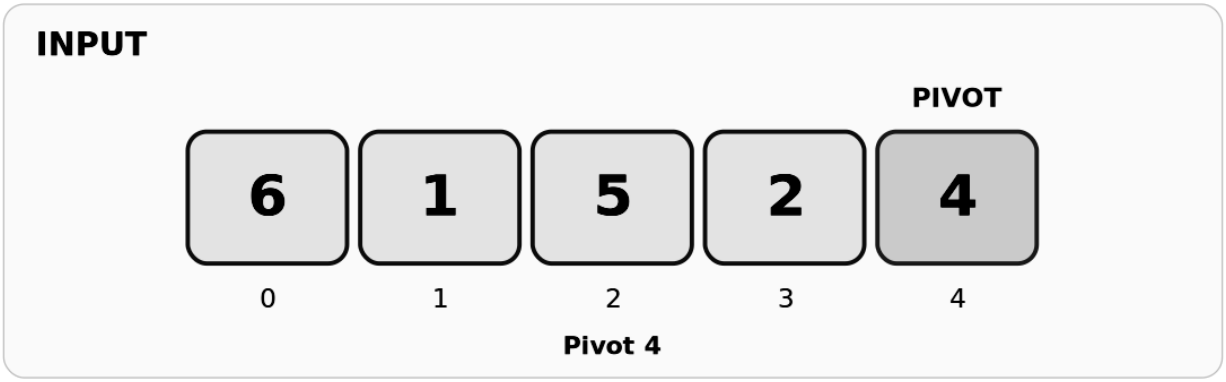
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace recursive quicksort on [6,1,5,2,4].

Partition around pivot 4, producing [1,2,4,6,5].



INPUT: 6, 1, 5, 2, 4. Pivot 4

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict the next state Partition around pivot 4, producing [1,2,4,6,5].

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Why is if(low>=high) return; required in a recursive C# quicksort?

Question 2

Use last-element Lomuto quicksort on [4,1,6,2]. Give the first partition and the final sorted array.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Describe the average and worst-case time complexity of quicksort, why these can differ, and one implementation trade-off relative to merge sort.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

After partition, the recursive calls should include the pivot index again.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

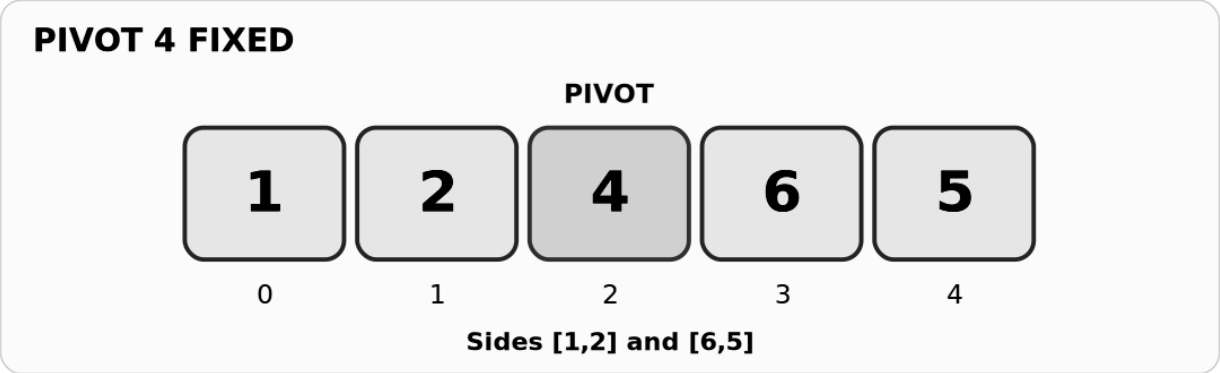
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace recursive quicksort on [6,1,5,2,4].

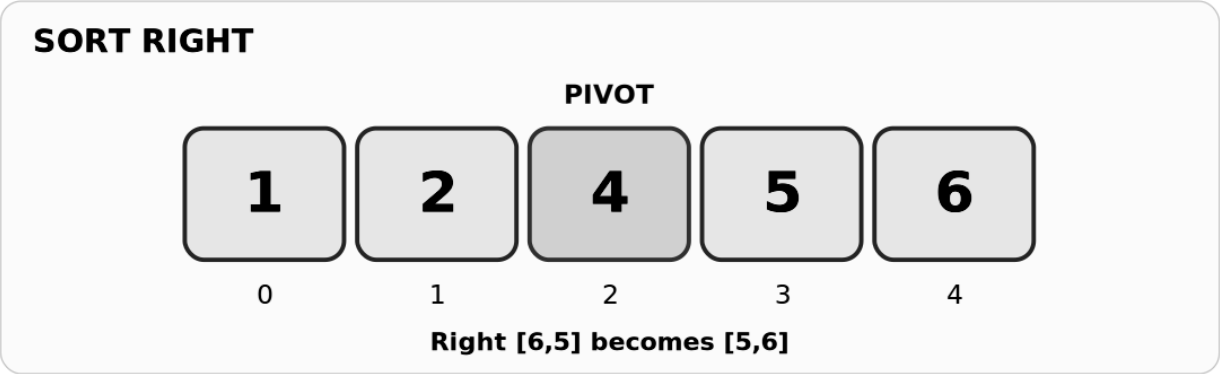
Partition around pivot 4, producing [1,2,4,6,5].



PIVOT 4 FIXED: 1, 2, 4, 6, 5. Sides [1,2] and [6,5]

Trace recursive quicksort on [6,1,5,2,4].

Partition around pivot 4, producing [1,2,4,6,5].



SORT RIGHT: 1, 2, 4, 5, 6. Right [6,5] becomes [5,6]

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Reveal and explain

The left [1,2] is already sorted; the right [6,5] needs another partition.

After sorting the right part, final array is [1,2,4,5,6].

IMPROVE: Explain one correction to your own reasoning.